

# gurps-ink

Typeset unofficial *GURPS* material  
Version 0.1.0

## Contents

|      |                                        |    |
|------|----------------------------------------|----|
| 1    | Introduction .....                     | 1  |
| 2    | Tutorial: make a sourcebook page ..... | 1  |
| 3    | How-to guides .....                    | 2  |
| 3.1  | Add the legal notices .....            | 2  |
| 3.2  | Write dice and damage .....            | 3  |
| 3.3  | Refer to GURPS books .....             | 3  |
| 3.4  | Make a stat block .....                | 3  |
| 3.5  | Keep your characters in one file ..... | 4  |
| 3.6  | Quote a character in the text .....    | 4  |
| 3.7  | Hide point costs .....                 | 4  |
| 3.8  | Check a point budget .....             | 5  |
| 3.9  | Keep a stat block in one column .....  | 5  |
| 3.10 | Change how a stat block looks .....    | 5  |
| 3.11 | Make your own layout .....             | 6  |
| 3.12 | Fix an error .....                     | 6  |
| 4    | What gurps-ink calculates .....        | 7  |
| 5    | Reference .....                        | 8  |
| 5.1  | Text and dice .....                    | 8  |
| 5.2  | Damage .....                           | 11 |
| 5.3  | Characters .....                       | 11 |
| 5.4  | Stat blocks .....                      | 19 |
| 6    | Legal .....                            | 21 |

## 1 Introduction

Use `gurps-ink` to write an unofficial *GURPS* sourcebook, adventure or handout in Typst. It typesets stat blocks and calculates their point costs. It writes dice, damage and book references in the style of the Steve Jackson Games books. It also gives you the notices that the Steve Jackson Games online policy asks for.

This manual has four parts:

- **Tutorial** (Section 2): make one sourcebook page, step by step. Start here if `gurps-ink` is new to you.
- **How-to guides** (Section 3): do one task, such as “hide point costs”.
- **What `gurps-ink` calculates** (Section 4): know which numbers come from `gurps-ink` and which come from you.
- **Reference** (Section 5): all functions and their arguments.

In each example, the code is on the left and the result is on the right. Each example in the tutorial and the how-to guides is complete: copy it into a new file and it compiles. A reference example needs the import line first.

## 2 Tutorial: make a sourcebook page

This tutorial makes the page on the right. It has a heading, text with a book reference, two stat blocks and the legal notices.

```
#import "@preview/gurps-ink:0.1.0": *

#set page(width: 11cm, height: 8.5cm, margin: 7mm, columns: 2)
#set text(size: 7pt)
#set par(justify: true)

#let npcs = (
  sergeant: character(
    name: "Sergeant Mara Venn",
    st: 12,
    dx: 11,
    ht: 11,
    advantage("Combat Reflexes", points: 15),
    disadvantage("Duty (City Watch)", points: -10),
    skill("Broadsword", 14, "DX/A"),
    skill("Shield", 13, "DX/E"),
    skill("Intimidation", 11, "Will/A"),
    melee-attack("Broadsword", 14, [#dice(.swing(12)) cut],
    reach: "1"),
  ),
  watchman: character(
    name: "Watchman",
    st: 11,
    skill("Spear", 12, "DX/A"),
    skill("Streetwise", 10, "IQ/A"),
    melee-attack("Spear", 12, [#dice(1, 1) imp], reach: "1"),
  ),
)

#place(top, float: true, scope: "parent") [= The Watch House]

The Watch keeps order in the lower city. A hero
who talks back to a guard rolls a Quick Contest
(#gurps-book("Basic Set", 348)). Sergeant Venn
swings her broadsword for #dice(.npcs.sergeant.sw) cut.

#block(breakable: false, stat-block(npcs.sergeant))

At night, two watchmen guard the door. They are
bored and easy to bribe.

#block(breakable: false, stat-block(npcs.watchman))

#text(size: 5pt)[#sjgames-disclaimer #sjgames-notice]
```

## The Watch House

The Watch keeps order in the lower city. A hero who talks back to a guard rolls a Quick Contest (*GURPS Basic Set* p. 348). Sergeant Venn swings her broadsword for 1d+2 cut.

**Sergeant Mara Venn** (75 points)  
ST 12 [20]; DX 11 [20]; IQ 10 [0]; HT 11 [10].  
HP 12 [0]; Will 10 [0]; Per 10 [0]; FP 11 [0].  
Basic Speed 5.50 [0]; Basic Move 5 [0]; Dodge 8.

SM 0; DR 0; thr 1d-1; sw 1d+2.  
**Advantages:** Combat Reflexes [15].  
**Disadvantages:** Duty (City Watch) [-10].  
**Skills:** Broadsword-14 [12]; Intimidation-11 [4]; Shield-13 [4].

### Attacks:

**Broadsword (14):** 1d+2 cut. Reach 1.

At night, two watchmen guard the door. They are bored and easy to bribe.

**Watchman** (20 points)

ST 11 [10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].  
HP 11 [0]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.

SM 0; DR 0; thr 1d-1; sw 1d+1.

**Skills:** Spear-12 [8]; Streetwise-10 [2].

### Attacks:

**Spear (12):** 1d+1 imp. Reach 1.

The material presented here is my original creation, intended for use with the *GURPS* system from Steve Jackson Games. This material is not official and is not endorsed by Steve Jackson Games. *GURPS* is a registered trademark of Steve Jackson Games, and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the SJ Games online policy.

Do these steps:

1. Import gurps-ink on the first line. All its functions are then available.
2. Set up the page. This page has two columns, as many sourcebooks have.
3. Make the characters with character(). Put them in one dictionary, npcs. Then you can use each character again in the text.
4. Write the text. #gurps-book("Basic Set", 348) refers to a page of the Basic Set. #dice(.npcs.sergeant.sw) writes the swing damage of the sergeant.
5. Typeset each character with stat-block(). Put each stat block in block(breakable: false, ..). Then a column break cannot split a stat block.
6. Add the legal notices at the end. Make them small with text(size: ..).

Next, read the how-to guides for the tasks that you need.

## 3 How-to guides

### 3.1 Add the legal notices

The Steve Jackson Games online policy tells you which notices your material must have. Read the policy before you publish. gurps-ink gives you the text of three notices. It does not give legal advice.

```
#import "@preview/gurps-ink:0.1.0": *

#set text(size: 7pt)
#sjgames-disclaimer

#sjgames-notice

#sjgames-game-aid[Jane Doe]
```

The material presented here is my original creation, intended for use with the **GURPS** system from [Steve Jackson Games](#). This material is not official and is not endorsed by Steve Jackson Games.

**GURPS** is a registered trademark of [Steve Jackson Games](#), and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the [SJ Games online policy](#).

**GURPS** is a trademark of [Steve Jackson Games](#), and its rules and art are copyrighted by [Steve Jackson Games](#). All rights are reserved by Steve Jackson Games. This game aid is the original creation of Jane Doe and is released for free distribution, and not for resale, under the permissions granted in the [Steve Jackson Games Online Policy](#).

- sjgames-disclaimer says that the material is yours and not official.
- sjgames-notice is the trademark and copyright notice.
- sjgames-game-aid is the notice for a free game aid. Give it the name of the author.

Put the notices on the title page or on the last page.

### 3.2 Write dice and damage

Use `dice(count, modifier)`. The result does not break across lines, and a negative modifier has a true minus sign. To write basic damage, spread `thrust(st)` or `swing(st)` into `dice`.

```
#import "@preview/gurps-ink:0.1.0": *

Roll #dice(3) against your skill.
The trap does #dice(2, -1) cr.
A ST 13 sword does
#dice(..swing(13)) cut.
A spell does #dice(1, $n$) burn.
```

Roll 3d against your skill. The trap does 2d–1 cr. A ST 13 sword does 2d–1 cut. A spell does 1d+*n* burn.

### 3.3 Refer to GURPS books

Use `gurps-book(title, pages)`. Do not write “GURPS” in the title: the function adds it. If you refer to a book many times, make a short function for it.

```
#import "@preview/gurps-ink:0.1.0": *

#let basic(..pages) = gurps-book(
  "Basic Set", ..pages,
)

See #gurps-book("Magic", 14),
#basic(16) and
#basic((170, 344)).
```

See *GURPS Magic* p. 14, *GURPS Basic Set* p. 16 and *GURPS Basic Set* pp. 170, 344.

### 3.4 Make a stat block

Make the character with `character()`. Give the attributes as named arguments. Give the traits, skills, spells and attacks as positional arguments, in any order. Then typeset the character with `stat-block()`.

```
#import "@preview/gurps-ink:0.1.0": *

#let rat = character(
  name: "Giant Rat",
  st: 6, dx: 12, ht: 12,
  sm: -1,
  advantage("Night Vision",
    points: 5, level: 5),
  skill("Brawling", 13, "DX/E"),
  melee-attack("Bite", 13,
    [#dice(1, -4) cut], reach: "C"),
)
#stat-block(rat)
```

**Giant Rat** (27 points)  
 ST 6 [-40]; DX 12 [40]; IQ 10 [0]; HT 12 [20].  
 HP 6 [0]; Will 10 [0]; Per 10 [0]; FP 12 [0].  
 Basic Speed 6.00 [0]; Basic Move 6 [0]; Dodge 9.  
 SM -1; DR 0; thr 1d-4; sw 1d-3.  
**Advantages:** Night Vision 5 [5].  
**Skills:** Brawling-13 [2].  
**Attacks:**  
**Bite (13):** 1d-4 cut. Reach C.

Give the point cost of each advantage and disadvantage with `points:`. If you do not, the stat block shows [?] and the total counts it as 0.

### 3.5 Keep your characters in one file

Put all characters in one dictionary. Then each chapter can use the same characters. When you change a character, each chapter shows the change.

```
#import "@preview/gurps-ink:0.1.0": *

#let npcs = (
  rat: character(name: "Giant Rat",
    st: 6),
  ogre: character(name: "Ogre",
    st: 20),
)

The ogre has
#level-of(npcs.ogre, "HP") HP.
The rat has
#level-of(npcs.rat, "HP") HP.
```

The ogre has 20 HP. The rat has 6 HP.

Put the import line and the `#let npcs = (..)` lines in a file, for example `npcs.typ`. In each chapter, write `#import "npcs.typ": npcs.`

### 3.6 Quote a character in the text

Use `level-of(char, name)` to get the level of an attribute, skill or spell. Use `char.thr` and `char.sw` to get the damage.

```
#import "@preview/gurps-ink:0.1.0": *

#let ogre = character(
  st: 20,
  skill("Brawling", 12, "DX/E"),
)

The ogre has
#level-of(ogre, "HP") HP and
Brawling-#level-of(ogre, "Brawling").
It punches for #dice(..ogre.thr) cr.
```

The ogre has 20 HP and Brawling-12. It punches for 2d-1 cr.

### 3.7 Hide point costs

Players do not need point costs in a handout or a bestiary. Use `show-points: false`.

```
#import "@preview/gurps-ink:0.1.0": *

#stat-block(
  character(
    name: "Bandit",
    st: 11,
    skill("Shortsword", 12, "DX/A"),
  ),
  show-points: false,
)
```

#### Bandit

ST 11; DX 10; IQ 10; HT 10.  
 HP 11; Will 10; Per 10; FP 10.  
 Basic Speed 5.00; Basic Move 5; Dodge 8.  
 SM 0; DR 0; thr 1d-1; sw 1d+1.  
**Skills:** Shortsword-12.

### 3.8 Check a point budget

For a pregenerated character or a template, use `assert` with `total-points`. If the character costs too much, compilation stops with your message.

```
#import "@preview/gurps-ink:0.1.0": *

#let squire = character(
  name: "Squire",
  st: 11, dx: 12,
  skill("Broadsword", 13, "DX/A"),
)
#assert(
  total-points(squire) <= 100,
  message: "Squire is over budget",
)
#stat-block(squire)
```

#### Squire (54 points)

ST 11 [10]; DX 12 [40]; IQ 10 [0]; HT 10 [0].  
 HP 11 [0]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
 Basic Speed 5.50 [0]; Basic Move 5 [0]; Dodge 8.  
 SM 0; DR 0; thr 1d-1; sw 1d+1.  
**Skills:** Broadsword-13 [4].

### 3.9 Keep a stat block in one column

A stat block can break across columns and pages. To prevent this, put it in a block that cannot break.

```
#import "@preview/gurps-ink:0.1.0": *

#block(
  breakable: false,
  stat-block(character(name: "Guard")),
)
```

#### Guard (0 points)

ST 10 [0]; DX 10 [0]; IQ 10 [0]; HT 10 [0].  
 HP 10 [0]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
 Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.  
 SM 0; DR 0; thr 1d-2; sw 1d.

### 3.10 Change how a stat block looks

Use the hooks of `stat-block()`. The `title` hook makes the first line. The `section` hook makes each labelled list, such as “Skills”.

```
#import "@preview/gurps-ink:0.1.0": *

#stat-block(
  character(
    name: "Guard",
    st: 11,
    skill("Spear", 12, "DX/A"),
  ),
  title: (char, total) => smallcaps(
    [#char.name, #total points],
  ),
  section: (label, body) => [
    #emph(label) --- #body
  ],
)
```

GUARD, 18 POINTS  
 ST 11 [10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].  
 HP 11 [0]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
 Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.  
 SM 0; DR 0; thr 1d-1; sw 1d+1.  
*Skills* — Spear-12 [8].

To frame a stat block, put it in a block with a stroke.

### 3.11 Make your own layout

For a different layout, such as a cast list, write a function. Give it the dictionary from `character()`. The reference for `character()` (Section 5) shows all keys of this dictionary.

```
#import "@preview/gurps-ink:0.1.0": *

#let one-line(char) = {
  let attributes = ("ST", "DX", "IQ", "HT")
  .map(k => [#k
#char.attributes.at(k).level])
  let skills = char.skills
  .map(s => [#s.name\-#s.level])
  [*#char.name: * #attributes.join[, ],
  _Skills:_ #skills.join[, ],]
}

#one-line(character(
  name: "Guard", st: 11,
  skill("Spear", 12, "DX/A"),
))
```

**Guard:** ST 11, DX 10, IQ 10, HT 10. *Skills:* Spear-12.

`char.skills.first().points` has the point cost of the first skill. `total-points(char)` has the total.

When Typst gets user-defined elements, `stat-block` will become one. Then show and set rules can change it, as they change built-in elements.

### 3.12 Fix an error

When an input is wrong, compilation stops. The error message tells you what is wrong. This table shows the frequent messages.

| The message contains                   | Do this                                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| is too low to buy                      | Make the skill level higher, or give the points as a number.                                                |
| cannot work out the cost of            | Make sure that the base ("DX" in "DX/A") is the name of an attribute, skill or spell of the same character. |
| more than one skill or spell is called | Give each skill and spell a different name.                                                                 |
| got unknown argument(s)                | Make sure that you spell the argument correctly.                                                            |
| covers ST 1 to 100                     | Give <code>thr:</code> and <code>sw:</code> to <code>character()</code> .                                   |

|                                 |                                                                   |
|---------------------------------|-------------------------------------------------------------------|
| points must be zero or negative | Give a disadvantage a negative cost, for example points :<br>-10. |
|---------------------------------|-------------------------------------------------------------------|

## 4 What gurps-ink calculates

character() calculates these values from the Basic Set rules:

- The point cost of each attribute and secondary characteristic.
- The point cost of each skill and spell that has a cost such as "DX/A".
- Dodge, from Basic Speed.
- Thrust and swing damage, from ST.
- The point total.

You must give these values:

- The point cost of each advantage and disadvantage.
- The skill level and damage of each attack.
- ST damage above ST 100. Give thr: and sw:.
- Discounts, such as the SM discount to ST. Give the cost yourself, for example st: (level: 20, points: 90).

A value that you give always replaces the calculated value.

## 5 Reference

### 5.1 Text and dice

#### 5.1.1 dice

Dice in the notation of the GURPS books. The result does not break across lines.

```
#dice(3) \
#dice(2, 1) \
#dice(4, -1) \
#dice(1, 0)
```

```
3d
2d+1
4d-1
1d
```

The modifier is optional. A zero modifier shows nothing. A negative modifier has a true minus sign. A content modifier, such as \$m\$, gets a +.

To write basic damage, spread `thrust()` or `swing()` into it: `#dice(..thrust(13))`.

##### 5.1.1.1 Parameters

```
dice(
  count: int content,
  ..modifier: int content
) -> content
```

###### 5.1.1.1.1 count int or content

The number of dice: an integer of 0 or more, or content such as \$n\$.

###### 5.1.1.1.2 ..modifier int or content

The modifier. It is optional; the default is 0.

### 5.1.2 gurps-book

A reference to a GURPS book in the house style of SJ Games: the title in bold italics, then the pages if you give them.

```
#gurps-book("High Tech") \
#gurps-book("Zombies", 3) \
#gurps-book("Warehouse 23", "1, 3-5") \
#gurps-book("Basic Set", (16, 170))
```

```
GURPS High Tech
GURPS Zombies p. 3
GURPS Warehouse 23 pp. 1, 3-5
GURPS Basic Set pp. 16, 170
```

One page (an int, or a string of digits) gets “p.”. All other pages get “pp.”. Commas join the pages in an array.

### 5.1.2.1 Parameters

```
gurps-book(  
  title: str content,  
  ..pages: int str content array  
) -> content
```

#### 5.1.2.1.1 title str or content

The title of the book, without “GURPS”.

#### 5.1.2.1.2 ..pages int or str or content or array

The page or pages. They are optional.

### 5.1.3 sjgames-game-aid

The notice from the SJ Games online policy for a free game aid that has no official license.

```
#sjgames-game-aid[Jane Doe]
```

***GURPS*** is a trademark of [Steve Jackson Games](#), and its rules and art are copyrighted by [Steve Jackson Games](#). All rights are reserved by Steve Jackson Games. This game aid is the original creation of Jane Doe and is released for free distribution, and not for resale, under the permissions granted in the [Steve Jackson Games Online Policy](#).

### 5.1.3.1 Parameters

```
sjgames-game-aid(author: str content) -> content
```

#### 5.1.3.1.1 author str or content

The author of the game aid.

### 5.1.4 gurps content

The name ***GURPS*** in bold italics. The SJ Games online policy asks for this style. Use it as you use other content.

`#gurps` is a generic, universal roleplaying system.

**GURPS** is a generic, universal roleplaying system.

To link it to the GURPS home page, put it in link: `#link("http://www.sjgames.com/gurps/", gurps)`.

#### 5.1.5 sjgames content

The name “Steve Jackson Games”.

`#gurps` is published by `#sjgames`.

**GURPS** is published by Steve Jackson Games.

#### 5.1.6 sjgames-disclaimer content

The disclaimer from the SJ Games online policy. It says that the material is yours and not official.

`#sjgames-disclaimer`

The material presented here is my original creation, intended for use with the **GURPS** system from [Steve Jackson Games](#). This material is not official and is not endorsed by Steve Jackson Games.

#### 5.1.7 sjgames-notice content

The trademark notice from the SJ Games online policy. It does not mention art, because the package includes no SJ Games art.

`#sjgames-notice`

**GURPS** is a registered trademark of [Steve Jackson Games](#), and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the SJ Games [online policy](#).

## 5.2 Damage

### 5.2.1 thrust

The thrust damage for a ST, as a (count, modifier) pair. Spread it into `dice()`.

```
#thrust(10) \  
#dice(..thrust(10))
```

```
(1, -2)  
1d-2
```

It covers ST 1 to 100. Above ST 40, the Basic Set table has a row for each 5 ST; between two rows, the lower row applies. For a ST that is not in this range, compilation stops: give thr: to `character()`.

#### 5.2.1.1 Parameters

```
thrust(st: int) -> array
```

##### 5.2.1.1.1 st int

The striking strength.

### 5.2.2 swing

The swing damage for a ST, as a (count, modifier) pair. Spread it into `dice()`. It covers the same ST as `thrust()`.

```
#dice(..swing(10))
```

```
1d
```

#### 5.2.2.1 Parameters

```
swing(st: int) -> array
```

##### 5.2.2.1.1 st int

The striking strength.

## 5.3 Characters

### 5.3.1 advantage

An advantage.

```
#let afro = advantage("Natural afro",  
points: 1)  
#let magery = advantage("Magery", points:  
25, level: 2)  
#afro.points, #magery.level
```

```
1, 2
```

#### 5.3.1.1 Parameters

```
advantage(  
  name: str content,  
  points: int none,  
  level: int none  
) -> dictionary
```

#### 5.3.1.1.1 points `int` or `none`

The point cost. If it is none, the stat block shows [?] and the total counts it as 0.

Default: `none`

#### 5.3.1.1.2 level `int` or `none`

The level of an advantage that has levels, such as Magery 2.

Default: `none`

### 5.3.2 disadvantage

A disadvantage. If you give a point cost, it must be zero or negative.

```
#disadvantage("Bad Temper", points:  
-10).points
```

-10

#### 5.3.2.1 Parameters

```
disadvantage(  
    name: str content,  
    points: int none,  
    level: int none  
) -> dictionary
```

#### 5.3.2.1.1 points `int` or `none`

The point cost, zero or less. If it is none, the stat block shows [?] and the total counts it as 0.

Default: `none`

#### 5.3.2.1.2 level `int` or `none`

The level of a disadvantage that has levels.

Default: `none`

### 5.3.3 perk

A perk. Its cost is always 1 point.

#### 5.3.3.1 Parameters

```
perk(name: str content) -> dictionary
```

### 5.3.4 quirk

A quirk. Its cost is always -1 point.

#### 5.3.4.1 Parameters

```
quirk(name: str content) -> dictionary
```

### 5.3.5 skill

A skill at a given level.

The third argument is the cost. It is optional. Give it in one of two forms:

- The points, such as 4.
- The base and the difficulty, such as "DX/E" or "Per/Average". Then `character()` calculates the points.

The base can be an attribute, or a skill or spell of the same character ("Typst/H"). The match uses the plain text of skill and spell names, so `[_Typst_]` matches "Typst". If a skill is a base for other skills, give it a name that has plain text.

The difficulty is E, A, H, VH or W, or the full word (Easy, Average, Hard, Very Hard, Wildcard).

```
#skill("Stealth", 12, "DX/A") \  
#skill("Brawling", 14, 4) \  
#skill("Carousing", 10)
```

```
(  
  kind: "skill",  
  name: "Stealth",  
  level: 12,  
  points: none,  
  base: "DX",  
  difficulty: "A",  
)  
(  
  kind: "skill",  
  name: "Brawling",  
  level: 14,  
  points: 4,  
  base: none,  
  difficulty: none,  
)  
(  
  kind: "skill",  
  name: "Carousing",  
  level: 10,  
  points: none,  
  base: none,  
  difficulty: none,  
)
```

### 5.3.5.1 Parameters

```
skill(  
  name: str content,  
  level: int,  
  ..cost: int str  
) -> dictionary
```

#### 5.3.5.1.1 level int

The skill level.

#### 5.3.5.1.2 ..cost int or str

The cost: points, or "<base>/<difficulty>". It is optional.

### 5.3.6 spell

A spell. It has the same arguments as `skill()`.

```
#spell("Fireball", 14, "IQ/H")
```

```
(  
  kind: "spell",  
  name: "Fireball",  
  level: 14,  
  points: none,  
  base: "IQ",  
  difficulty: "H",  
)
```

#### 5.3.6.1 Parameters

```
spell(  
  name: str content,  
  level: int,  
  ..cost: int str  
) -> dictionary
```

### 5.3.7 melee-attack

A melee attack.

```
#melee-attack("Punch", 18, [#dice(1, -1)
cr], reach: "C")
```

```
(
  kind: "melee-attack",
  name: "Punch",
  level: 18,
  damage: sequence(box(body: sequence([1,
[d], [-1]]), [ ], [cr])),
  notes: none,
  reach: "C",
)
```

### 5.3.7.1 Parameters

```
melee-attack(
  name: str content,
  level: int,
  damage: str content,
  reach: str content none,
  notes: str content none
) -> dictionary
```

#### 5.3.7.1.1 level int

The skill level with the attack.

#### 5.3.7.1.2 damage str or content

The damage, such as [#dice(2, 1) cut].

#### 5.3.7.1.3 notes str or content or none

Notes. The stat block shows them after the attack.

Default: none

### 5.3.8 ranged-attack

A ranged attack.

```
#ranged-attack("Bow", 14, [#dice(1, 2) imp],
range: "150/200")
```

```
(
  kind: "ranged-attack",
  name: "Bow",
  level: 14,
  damage: sequence(box(body: sequence([1,
[d], [+2]]), [ ], [imp])),
  notes: none,
  range: "150/200",
)
```

### 5.3.8.1 Parameters

```
ranged-attack(  
  name: str content,  
  level: int,  
  damage: str content,  
  range: str content none,  
  notes: str content none  
) -> dictionary
```

### 5.3.9 character

Makes a character. Give only the values that are not the default.

Give each attribute as a level. `character()` calculates its cost from the Basic Set rules. To give a different cost, such as an SM discount to ST, give a dictionary: `st: (level: 20, points: 90)`.

Give the traits, skills, spells and attacks as positional arguments.

```
#let napoleon = character(  
  name: "Napoleon",  
  st: 9, hp: 12,  
  advantage("Natural afro", points: 1),  
  quirk("Big teeth"),  
  skill("Nunchuck", 16, "DX/E"),  
)  
#napoleon.attributes.HP \  
#total-points(napoleon) points
```

```
(level: 12, points: 6)  
16 points
```

The result is a dictionary. Its keys are public API. Read them, or use `level-of()`, to quote numbers or to make your own layout. The keys are:

- kind: "character"; name.
- attributes: keyed ST, DX, IQ, HT, HP, Will, Per, FP, Basic Speed, Basic Move, in that order; each (level:, points:).
- dodge, sm, dr, thr, sw: as given or calculated.
- advantages, perks, disadvantages, quirks: arrays of (kind:, name:, points:, level:) in the order given.
- skills, spells: arrays of (kind:, name:, level:, points:, base:, difficulty:). If the skill has a base, points is calculated.
- attacks: arrays of (kind:, name:, level:, damage:, notes:) plus reach (melee) or range (ranged).

A value that is not known is none.

### 5.3.9.1 Parameters

```
character(  
  name: str content none ,  
  st: int dictionary ,  
  dx: int dictionary ,  
  iq: int dictionary ,  
  ht: int dictionary ,  
  hp: auto int dictionary ,  
  will: auto int dictionary ,  
  per: auto int dictionary ,  
  fp: auto int dictionary ,  
  basic-speed: auto int float dictionary ,  
  basic-move: auto int dictionary ,  
  dodge: auto int ,  
  sm: int ,  
  dr: int str content ,  
  thr: auto array content ,  
  sw: auto array content ,  
  ..traits: dictionary  
) -> dictionary
```

#### 5.3.9.1.1 name str or content or none

The name. The stat block shows it in the title.

Default: none

#### 5.3.9.1.2 basic-speed auto or int or float or dictionary

A multiple of 0.25.

Default: auto

#### 5.3.9.1.3 dodge auto or int

The default is [Basic Speed] + 3. It has no point cost.

Default: auto

#### 5.3.9.1.4 sm int

The Size Modifier.

Default: 0

#### 5.3.9.1.5 dr int or str or content

The Damage Resistance, such as 2 or [3 (torso only)].

Default: 0

#### 5.3.9.1.6 thr auto or array or content

The thrust damage: a (count, modifier) pair or content. The default is `thrust()` of ST.

Default: `auto`

#### 5.3.9.1.7 sw auto or array or content

The swing damage, as for thr. The default is `swing()` of ST.

Default: `auto`

#### 5.3.9.1.8 ..traits dictionary

Results of `advantage()`, `disadvantage()`, `perk()`, `quirk()`, `skill()`, `spell()`, `melee-attack()` and `ranged-attack()`, in any order.

### 5.3.10 level-of

The level of an attribute, skill or spell, found by its name. For a skill or spell, it uses the plain text of the name. It also knows "Dodge" and "SM".

Compilation stops if no name matches, or if two names match. The message lists the names that the character has.

```
#let c = character(dx: 12, skill("Stealth",  
13, "DX/A"))  
#level-of(c, "Basic Speed"), #level-of(c,  
"Stealth")
```

5.5, 13

#### 5.3.10.1 Parameters

```
level-of(  
  char: dictionary ,  
  name: str  
) -> int float
```

##### 5.3.10.1.1 char dictionary

A result of `character()`.

##### 5.3.10.1.2 name str

The name, such as "ST", "Basic Move" or "Stealth".

### 5.3.11 total-points

The point total of a character: the sum of all known costs. A trait that has no cost counts as 0.

```
#total-points(character(st: 12, dx: 11))
```

40

### 5.3.11.1 Parameters

```
total-points(char: dictionary) -> int
```

#### 5.3.11.1.1 char dictionary

A result of `character()`.

## 5.4 Stat blocks

### 5.4.1 stat-block

Typesets a character from `character()` as a stat block, in the style of the SJ Games books.

The lines are: the title (name and point total), ST/DX/IQ/HT, HP/Will/Per/FP, Basic Speed/Basic Move/Dodge and SM/DR/thr/sw. Then come the lists: advantages, perks, disadvantages, quirks, skills, spells and attacks. Each list is in alphabetical order. An empty list does not show.

```
#stat-block(character(  
  name: "Napoleon",  
  st: 9, hp: 12,  
  advantage("Natural afro", points: 1),  
  quirk("Big teeth"),  
  skill("Nunchuck", 16, "DX/E"),  
  melee-attack("Punch", 18, [#dice(5) cr],  
  reach: "C, 1",  
  notes: [Believe it!]),  
))
```

#### **Napoleon** (16 points)

ST 9 [-10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].  
HP 12 [6]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.  
SM 0; DR 0; thr 1d-2; sw 1d-1.

**Advantages:** Natural afro [1].

**Quirks:** Big teeth [-1].

**Skills:** Nunchuck-16 [20].

#### **Attacks:**

**Punch (18):** 5d cr. Reach C, 1. Believe it!

To frame it, put it in a block: `#block(stroke: 0.5pt, inset: 8pt)[#stat-block(c)]`.

Two hooks change parts of the stat block:

```
#stat-block(
  character(name: "Guard", st: 11,
skill("Spear", 12, "DX/A")),
  title: (char, total) =>
smallcaps[#char.name, #total points],
  section: (label, body) => [#emph(label)
--- #body],
)
```

GUARD, 18 POINTS  
ST 11 [10]; DX 10 [0]; IQ 10 [0]; HT 10 [0].  
HP 11 [0]; Will 10 [0]; Per 10 [0]; FP 10 [0].  
Basic Speed 5.00 [0]; Basic Move 5 [0]; Dodge 8.  
SM 0; DR 0; thr 1d-1; sw 1d+1.  
*Skills* — Spear-12 [8].

For a different layout, write your own function that uses the dictionary from `character()`. The manual shows how.

#### 5.4.1.1 Parameters

```
stat-block(
  char: dictionary,
  show-points: bool,
  title: auto none function,
  section: auto function
) -> content
```

##### 5.4.1.1.1 char dictionary

A result of `character()`.

##### 5.4.1.1.2 show-points bool

Show the point costs ([10]) and the total. Use `false` for a handout or a bestiary.

Default: `true`

##### 5.4.1.1.3 title auto or none or function

Makes the first line. The call is `title(char, total)`. `total` is the point total, or `none` if `show-points` is `false`. Return content or a string. To remove the line, return `none` or give `title: none`. `auto` is this function:

```
(char, total) => {
  let points = if total != none [#total points]
  if char.name == none { points }
  else [#strong(char.name)#if points != none [ (#points)]]
}
```

Default: `auto`

#### 5.4.1.1.4 section `auto` or `function`

Makes each list: Advantages, Perks, Disadvantages, Quirks, Skills, Spells and Attacks. The call is `section(label, body)`. `body` is the finished list: the entries, with semicolons between them and a full stop at the end. For attacks, `body` is a block with one paragraph for each attack. Return content or a string. To remove the list, return none. `auto` is `(label, body) => [#strong[#label:] #body]`.

Default: `auto`

## 6 Legal

The material presented here is my original creation, intended for use with the **GURPS** system from [Steve Jackson Games](#). This material is not official and is not endorsed by Steve Jackson Games.

**GURPS** is a registered trademark of [Steve Jackson Games](#), and is copyrighted by Steve Jackson Games. All rights are reserved by Steve Jackson Games. This material is used here in accordance with the [SJ Games online policy](#).